



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Specifica Tecnica_G

Versione	0.0.2
Stato	In corso
Redazione	Giulia Barzon
Verifica_G	Verificato
Approvazione	Da approvare
Proprietario	ByteMe
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin

Registro dei Cambiamenti

Versione	Data	Autore	Descrizione	Verifica _G
0.0.2	14/04/2026	Joseph Grant	Aggiunta tecnologie e architettura	
0.0.1	28/03/2026	Giulia Barzon	Creazione del documento, scrittura introduzione	Tommaso Tom- bacco

Tabella 1: Registro delle versioni del documento

Indice

1	Introduzione	1
1.1	Scopo del documento	1
1.2	Approccio alla redazione	1
1.3	Scopo del prodotto	1
1.4	Glossario _G	1
1.5	Riferimenti	1
1.5.1	Riferimenti informativi	2
1.6	Tecnologie	2
1.6.1	Framework frontend	2
1.6.2	Librerie frontend	3
1.6.3	Linguaggio frontend	3
1.6.4	Tecnologie di build	3
1.6.5	Tecnologie di test	4
1.6.6	Framework api _G backend	4
1.6.7	Linguaggio backend	4
1.6.8	Server backend	4
1.6.9	Tecnologie deployment	5
1.7	Architettura	5
1.7.1	Architettura backend	5
1.7.2	UML backend	5
1.8	Deployment	5
1.9	Architettura frontend	5
1.9.1	UML frontend	5
1.10		5

1 Introduzione

1.1 Scopo del documento

Il presente documento costituisce la **Specifica Tecnica_G** del prodotto *Second Brain*, proposto dall'azienda *Zucchetti* al gruppo ByteMe.

Il documento descrive in dettaglio:

- le **tecnologie** adottate (linguaggi, framework, librerie, strumenti);
- l'**architettura** logica e fisica del sistema, con i pattern architetturali scelti;
- i **design pattern** applicati a livello di componenti, con motivazione delle scelte;
- la **documentazione API_G** tra frontend e backend;
- i **requisiti soddisfatti** con riferimento al documento *Analisi dei Requisiti_G* v1.4.1.

Il documento è rivolto al team di sviluppo come guida implementativa e a committente e proponente come verifica_G della coerenza tra scelte progettuali e requisiti concordati.

1.2 Approccio alla redazione

Il documento viene redatto in modo **incrementale**, assicurando coerenza con gli sviluppi in corso. Non può essere considerato completo fino al rilascio della versione 1.0.0.

1.3 Scopo del prodotto

Il progetto *Second Brain* mira a sviluppare un editor di testo_G avanzato basato sul linguaggio Markdown_G e potenziato dall'integrazione di Large Language Model (LLM). L'applicazione web consentirà all'utente di:

- scrivere e visualizzare in tempo reale testi formattati in Markdown_G;
- interagire con un LLM per riassumere, correggere, riscrivere e tradurre testi;
- ottenere analisi critiche multi-prospettiche secondo il metodo dei *Sei Cappelli per Pensare* di Edward De Bono;
- delegare all'AI la stesura completa di un testo tramite Distant Writing_G.

1.4 Glossario_G

Per evitare ambiguità, i termini tecnici usati nel documento vengono definiti in modo più approfondito nel documento Glossario_G v2.0.0 che si può trovare nel sito web del gruppo ByteMe alla sezione Documenti Interni e raggiungibile dal seguente link:

ByteMe/Glossario_G v2.0.0

I termini che vengono considerati meritevoli di approfondimenti sono indicati con una G a pedice.

1.5 Riferimenti

- *Norme di Progetto_G* versione 2.0.0
ByteMe/Norme di Progetto_G v2.0.0
- Capitolato d'appalto C6: *Second Brain* (Zucchetti S.p.A.)
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
- Regolamento del progetto didattico:
<https://www.math.unipd.it/~tullio/IS-1/2023/Dispense/PD2.pdf>

1.5.1 Riferimenti informativi

- *Analisi dei Requisiti*_G versione 2.0.0 (Ultimo accesso: 01/04/2026)
PB - Analisi dei Requisiti_G v2.0.0
- *Piano di Qualifica* versione 1.0.0 (Ultimo accesso: 01/04/2026)
PB - Piano di Qualifica v2.0.0
- Verbali interni ed esterni
- Glossario_G v2.0.0 (Ultimo accesso: 01/04/2026)
Documenti interni - Glossario_G v2.0.0
- Repository_G ufficiale di EasyMDE_G: <https://github.com/Ionaru/easy-markdown-editor>
(Ultimo accesso: 24/03/2026)
- Documentazione linguaggio markdown_G - CommonMark: <https://commonmark.org/>
(Ultimo accesso: 20/02/2026)
- Documentazione React_G 19: <https://react.dev>
Ultimo accesso: 27/03/2026
- Principi del Bulletproof React_G (repository_G ufficiale): <https://github.com/alan2207/bulletproof-react>
(Ultimo accesso: 27/03/2026)
- Documentazione Flask_G: <https://flask.palletsprojects.com>
(Ultimo accesso: 10/03/2026)
- Documentazione Docker_G: <https://docs.docker.com>
(Ultimo accesso: 27/02/2026)
- Documentazione TypeScript: <https://www.typescriptlang.org/docs/>
(Ultimo accesso: 27/03/2026)
- Slide del corso di Ingegneria del Software
(Ultimo accesso: 27/03/2026)
- Principi SOLID (slide prof. Cardin)
- Pattern architetturali (slide prof. Cardin)

1.6 Tecnologie

1.6.1 Framework frontend

Nome	versione	Descrizione
React _G	19.2.0	Libreria JavaScript per la costruzione di interfacce utente basate su componenti riutilizzabili.

Tabella 2: Descrizione del framework frontend

1.6.2 Librerie frontend

Nome	versione	Descrizione
EasyMDE _G	2.20.0	Editor Markdown _G con anteprima in tempo reale, basato su CodeMirror.
Axios	1.7.9	Client HTTP basato su Promise per effettuare richieste REST dal browser e da Node.js.
TanStack Query	-	Libreria per la gestione del fetching, caching e sincronizzazione dello stato server in React _G .
CSS Modules _G	-	Sistema di scope locale per i fogli di stile CSS, che evita conflitti tra classi in componenti diversi.
React _G Hot Toast	-	Libreria leggera per la visualizzazione di notifiche toast personalizzabili in React _G .
eslint	9.31.1	Strumento di linting per JavaScript e TypeScript, che aiuta a mantenere uno stile di codice coerente e a individuare errori.

Tabella 3: Descrizione delle librerie frontend

1.6.3 Linguaggio frontend

Nome	versione	Descrizione
TypeScript	-	Superset tipizzato di JavaScript che aggiunge tipizzazione statica e migliora la manutenibilità del codice.
TSX	-	Estensione di TypeScript che consente di scrivere markup JSX _G all'interno di file tipizzati, usata per i componenti React _G .

Tabella 4: Descrizione dei linguaggi frontend

1.6.4 Tecnologie di build

Nome	versione	Descrizione
Vite _G	7.3.1	Build tool moderno e veloce per applicazioni web, con supporto nativo a ES modules e Hot Module Replacement.

Tabella 5: Descrizione delle tecnologie di build

1.6.5 Tecnologie di test

Nome	versione	Descrizione
Vitest	4.1.4	Framework di testing per applicazioni Vite _G , compatibile con l'API _G di Jest e ottimizzato per TypeScript.
Pytest	7.0.0	Framework di testing per Python, con sintassi semplice e supporto a fixture, parametrizzazione e plugin.
testing library React _G	16.3.2	Libreria per il testing di componenti React _G , che fornisce utilities per interagire con i componenti in modo simile a come un utente reale lo farebbe.

Tabella 6: Descrizione delle tecnologie di test

1.6.6 Framework api_G backend

Nome	versione	Descrizione
Flask _G	-	Microframework Python per lo sviluppo di API _G REST, leggero ed estensibile tramite plugin.

Tabella 7: Descrizione del framework api_G backend

1.6.7 Linguaggio backend

Nome	versione	Descrizione
Python	3.12-slim	Linguaggio di programmazione ad alto livello, utilizzato per la logica di business e l'implementazione delle API _G .

Tabella 8: Descrizione del linguaggio backend

1.6.8 Server backend

Nome	versione	Descrizione
-	-	-

Tabella 9: Descrizione del server backend

1.6.9 Tecnologie deployment

Nome	versione	Descrizione
Docker _G	-	Piattaforma per la containerizzazione delle applicazioni, che garantisce ambienti riproducibili e portabili.

Tabella 10: Descrizione delle tecnologie di deployment

1.7 Architettura

1.7.1 Architettura backend

1.7.2 UML backend

Per lo sviluppo del backend, abbiamo deciso di utilizzare un'architettura a monolite, che comunica in modalità REST con il frontend. Abbiamo usato una struttura layered a tre livelli per il monolite:

- **Presentation Layer:** ricezione delle richieste HTTP, controlli sulla validità dei dati.
- **Business Layer:** Costruzione del prompt_G, selezione del modello IA.
- **Data Layer:** Gestione delle chiamate ai modelli IA.

1.8 Deployment

Per il deployment abbiamo scelto di usare Docker_G, che ci permette di creare container isolati per il frontend e il backend, garantendo coerenza tra gli ambienti di sviluppo e produzione. Ogni componente ha un proprio Dockerfile che definisce le dipendenze e la configurazione necessaria per l'esecuzione.

1.9 Architettura frontend

1.9.1 UML frontend

Per il frontend, abbiamo adottato un'architettura MVVM (Model-View-ViewModel) che si adatta bene al paradigma dei componenti di React_G.

1.10